

Document Number: P0981R0

Date: 2018-03-18

Audience: Evolution

Revises: none

Reply to: Richard Smith (richardsmith@google.com), Gor Nishanov (gorn@microsoft.com)

Halo: coroutine Heap Allocation eLision Optimization: the joint response

Summary

During the discussion of coroutines in a Tuesday session in Jacksonville 2018, we had the following exchange:

Richard Smith: I have concerns about the generator example. ... The optimization relies on `accumulate` being inlinable or optimizable. ... You need to inline at least `begin` to know that the value of the handle is not changed.

Gor Nishanov: Let's take this offline and come back with the joint response.

This document is that joint response. It evaluates the necessary conditions for heap allocation elision to work. After careful study, we find that:

1. Heap allocation elision optimization DOES NOT require inlining a potentially unbounded amount of code, such as the coroutine body OR the algorithms using the coroutine (such as `accumulate` in the generator example).
2. It DOES REQUIRE inlining some compiler synthesized code (such as the coroutine ramp) and inlining a bounded amount of wrapper/glue code that gives the coroutine the desired semantics (such as `begin` in the generator example). Specifically, we need to inline (or analyze interprocedurally) sufficient code to conclude that all execution paths that create the coroutine and then leave the calling function will include a call to `destroy` on a `coroutine_handle` denoting the coroutine (typically in the destructor of the coroutine type).
3. With the implementation of coroutines in clang today, inlining the coroutine ramp function requires the coroutine to be defined in the same translation unit. We believe that alternative implementation strategies may allow this optimization to apply across ABI boundaries when the definition of the coroutine is not available in the current translation unit, but at this time have not analyzed the tradeoffs of such approaches.

The rest of the document goes over details of what exactly needs to be inlined and explores generator and task examples.

Terminology

Coroutine frame: a memory location where state of the coroutine that has to be preserved across suspend points is stored. The coroutine frame can reside on the heap or on the stack, depending on whether coroutine heap elision is in effect or whether a custom allocator is provided by the user to override default allocation of the coroutine state.

User-authored coroutine body: the *function-body* of the coroutine.

Coroutine state machine: a compiler created transformation of the *user-authored coroutine body*

Coroutine ramp function: a compiler synthesized ramp/trampoline/thunk that creates an initial coroutine state and starts the coroutine state machine.

Coroutine handle: an object of a `coroutine_handle` type that points at a coroutine state machine.

Coroutine type: a wrapper around the coroutine handle that has the necessary bindings to give the coroutine the desired semantics. For example: a `generator<T>` or a `task<T>`.

Coroutine escaping its caller: an operation within the caller of the coroutine that could make the address of the object of coroutine type visible outside the caller, excluding such cases where the compiler can prove that such escaping does not occur. For example: storing the address of the object of coroutine type into a global or through an externally-supplied pointer, or calling a member function on the object of coroutine type whose body is not available for examination (or is not examined due to optimizer limitations).

Case analysis

Generator case

Let's consider the example presented in P0978R0:

```
generator<int> range(int from, int to) { // user-authored coroutine body starts
    for (int i = from; i < to; ++i)
        co_yield i;
} // user authored coroutine body ends

int main() {
    auto s = range(1, 10);
    return std::accumulate(s.begin(), s.end(), 0);
}
```

Conceptually, the coroutine transformation will transform `range(int, int)` into something like this:

```
struct range$state-machine { ... transformed user-authored-body-is-here ... };

generator<int> range(int from, int to) { // coroutine ramp/thunk/trampoline
    auto s = new range$state-machine(from, to);
```

```

    return s->promise.get_return_object();
}

```

For the coroutine heap allocation elision optimization to work we need the following to be available for inlining:

- the coroutine ramp function
- `get_return_object()`
- `generator<int>::begin()`
- the constructor and move constructor of `generator<int>`
- the destructor of `generator<int>`
- `coroutine_handle<>::destroy` (either a compiler builtin or inlineable to a compiler builtin)

We DO NOT need to inline the body of the coroutine or the algorithm `accumulate`, unless the `iterator` type retains a pointer or reference to the `generator<int>` object.

Note that authors of coroutine types should be aware of this point: at least one blogger has described an implementation of `generator<T>::iterator` that holds a `generator<T>*`. However, every implementation of `generator<T>` we can find in the wild holds a `coroutine_handle<Promise>` instead, which avoids the problem, and allows heap allocation elision without inlining `accumulate`.

To give a feel of how big the functions that need to be inlined are, here is a typical generator implementation:

```

struct generator {
    struct promise_type { ... };

    struct iterator {
        ...
        coroutine_handle<promise_type> h_copy;
    };
    ...
    coroutine_handle<promise_type> h; // wrapper around void* trivial to construct
};

```

Constructors that need to be inlinable:

```

generator(coroutine_handle<promise_type> h) : h(h) {}
generator(generator&& rhs) : h(rhs.h) { rhs.h = nullptr; }

```

Destructor must be inlinable:

```

~generator() { if (h) h.destroy(); }

```

`get_return_object` must be inlinable:

```
generator promise_type::get_return_object() {  
    return {coroutine_handle::from_promise(*this)};  
}
```

`begin()` must be inlinable

```
iterator begin() {  
    if (h) h.resume();  
    return {h};  
}
```

Generator case with the Ranges TS

Under the Ranges TS, we expect that generators will be passed by reference to algorithms. If these algorithms extract the `begin` / `end` iterators from their ranges and then call other functions, only the range wrapper function need be inlined. For example, given:

```
template<InputRange Rng, class Proj = identity,  
        IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>  
bool all_of(Rng &&rng, Pred pred, Proj proj = Proj{}) {  
    return all_of(begin(rng), end(rng), ref(pred), ref(proj));  
}
```

Heap allocation elision optimization in `all_of(range(1, 5), pred)` only requires that the wrapper version of `all_of` be inlined in addition to those functions identified above. The algorithm implementation does not need to be inlined.

Task example

Let's consider the `big_task` example from P0978R0 with a small addition of a call to an executor (as the question was raised whether a coroutine posted to execute on an executor can have its allocation elided).

```
task<> subtask(Executor ex) {  
    ...  
    co_await execute_on(ex);  
    ...  
}  
  
task<> big_task(Executor ex) {  
    ...
```

```

    co_await subtask(ex); // subtask frame allocation elided
    ...
}

```

Similar to the generator case, only a small number of library functions need to be available for inlining:

- coroutine ramp function
- `get_return_object()`
- `await_suspend/await_ready/await_resume` of the task
- constructor and move constructor of `task<>`
- destructor of `task<>`
- `coroutine_handle<>::destroy` (either a compiler builtin or inlineable to a compiler builtin)

We DO NOT need to inline:

- executor `.execute/on_execute`
- body of `subtask` or `big_task`

To give a feel of how big the functions that need to be inlined are, here is a typical `task` implementation:

```

struct task {
    struct promise_type { ... };
    ...
    coroutine_handle<promise_type> h; // wrapper around void* trival to construct/
};

```

Constructors:

```

task(coroutine_handle<promise_type> h) : h(h) {}
task(task&& rhs) : h(rhs.h) { rhs.h = nullptr; }

```

`get_return_object`:

```

task promise_type::get_return_object() {
    return {coroutine_handle::from_promise(*this)}; }

```

Destructor:

```

~task() { if (h) h.destroy(); }

```

await_suspend:

```
auto await_suspend(coroutine_handle<> waiter) {  
    h.promise().waiter = waiter;  
    return h;  
}
```

await_ready/await_resume:

```
constexpr bool await_ready() const noexcept { return false; }  
constexpr void await_resume() const noexcept {}
```

Conclusions

We find that the functions that need to be inlined for coroutine heap allocation elision optimization to work are tiny and we would want them inlined anyway irrespective of whether they are needed for heap allocation elision to work or not. We anticipate that optimizers will reliably perform the optimization on code patterns similar to those above, but that allocations will be performed for code patterns where the object of coroutine type escapes or when compiling without optimization.

Some constraints are imposed on authors of coroutine types and on consumers of coroutine types. Code should avoid unnecessarily retaining pointers and references to the coroutine type, by extracting the necessary information from it in an inlineable wrapper. These constraints are straightforward to satisfy in the cases we have examined, but are novel.